

Отчёт по задаче «Города и дороги»

Смола Артём, группа 23.Б08

1 Постановка задачи

Дан связный неориентированный граф: города — вершины, дороги — рёбра с весами (длинами). Заданы k столиц, каждая столица соответствует отдельному государству. Требуется распределить все города между государствами по правилам:

1. Изначально в каждом государстве находится только его столица.
2. Далее государства ходят по очереди от 1 до k .
3. На ходе государство присоединяет один ближайший незанятый город, который имеет дорогу хотя бы к одному уже принадлежащему ему городу.

Нужно вывести для каждого государства список принадлежащих ему городов в порядке присоединения.

2 Принцип решения

Для каждого государства используется отдельная приоритетная очередь. Её элемент содержит:

- длину дороги;
- номер кандидата-города;
- город данного государства, из которого найден кандидат.

Алгоритм:

1. Считать граф списками смежности.
2. Пометить столицы как уже занятые соответствующими государствами.
3. Для каждой столицы добавить в её очередь все исходящие рёбра к незанятым городам.
4. Пока есть незанятые города, циклически пройти по государствам:
 - (a) удалить из вершины очереди устаревшие записи (если город уже занял кто-то другой);
 - (b) взять минимального кандидата;
 - (c) присоединить город к текущему государству;
 - (d) добавить в очередь рёбра из нового города к ещё незанятым городам.

Алгоритм полностью реализует правило поочерёдного присоединения ближайшего доступного города. При равенстве длины в очереди применяется правило разрешения равенств: сначала по меньшему номеру города, затем по меньшему номеру города-источника.

3 Корректность

Инвариант: после каждого присоединения у каждого государства его множество городов остается связным, поскольку новый город всегда присоединяется по ребру к уже принадлежащему городу. Цикл ходов строго следует правилу задачи: государства ходят по порядку от 1 до k . На каждом ходе выбирается минимальный допустимый кандидат из очереди текущего государства. Следовательно, порядок присоединений в программе совпадает с постановкой задачи.

4 Ограничения реализации

- входной граф должен быть связным;
- при несвязном графе программа завершает работу с ошибкой;
- длины дорог предполагаются неотрицательными.

5 Исходный код

Исходный код программы и тестовые данные доступны в репозитории GitHub: https://github.com/artem-smola/formal_lang.

6 Код программы

Код программы:

```
#include <algorithm>
#include <iostream>
#include <limits>
#include <queue>
#include <stack>
#include <tuple>
#include <utility>
#include <vector>

struct Edge {
    int to;
    int len;
};

struct Candidate {
    int len;
    int city;
    int from;
};

struct CandidateCmp {
    bool operator()(const Candidate& a, const Candidate& b) const {
        if (a.len != b.len) {
            return a.len > b.len;
        }
        if (a.city != b.city) {
            return a.city > b.city;
        }
        return a.from > b.from;
    }
};
```

```

    }
};

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);

    int n = 0;
    int m = 0;
    if (!(std::cin >> n >> m)) {
        std::cerr << "Ошибка: не удалось прочитать n и m.\n";
        return 1;
    }

    if (n <= 0 || m < 0) {
        std::cerr << "Ошибка: некорректные значения n или m.\n";
        return 1;
    }

    std::vector<std::vector<Edge>> graph(n + 1);
    for (int e = 0; e < m; ++e) {
        int u = 0;
        int v = 0;
        int len = 0;
        if (!(std::cin >> u >> v >> len)) {
            std::cerr << "Ошибка: не удалось прочитать дорогу на строке " << (e
                ↵ + 1) << ".\n";
            return 1;
        }

        if (u < 1 || u > n || v < 1 || v > n || len < 0) {
            std::cerr << "Ошибка: некорректные данные дороги на строке " << (e +
                ↵ 1) << ".\n";
            return 1;
        }

        graph[u].push_back({v, len});
        graph[v].push_back({u, len});
    }

    int k = 0;
    if (!(std::cin >> k)) {
        std::cerr << "Ошибка: не удалось прочитать число столиц k.\n";
        return 1;
    }
    if (k <= 0 || k > n) {
        std::cerr << "Ошибка: некорректное число столиц k.\n";
        return 1;
    }

    {
        std::vector<char> used(n + 1, 0);
        std::stack<int> st;
        st.push(1);
        used[1] = 1;

        while (!st.empty()) {
            const int v = st.top();
            st.pop();
            for (const auto& edge : graph[v]) {

```

```

        if (!used[edge.to]) {
            used[edge.to] = 1;
            st.push(edge.to);
        }
    }
}

for (int v = 1; v <= n; ++v) {
    if (!used[v]) {
        std::cerr << "Ошибка: граф дорог несвязный, корректное
        ↪ распределение невозможно.\n";
        return 1;
    }
}

std::vector<int> capitals(k + 1, 0);
std::vector<int> owner(n + 1, 0);
std::vector<std::vector<int>> states(k + 1);

for (int state = 1; state <= k; ++state) {
    int cap = 0;
    if (!(std::cin >> cap)) {
        std::cerr << "Ошибка: не удалось прочитать столицу для государства "
        ↪ << state << ".\n";
        return 1;
    }
    if (cap < 1 || cap > n) {
        std::cerr << "Ошибка: некорректный номер столицы.\n";
        return 1;
    }
    if (owner[cap] != 0) {
        std::cerr << "Ошибка: столица " << cap << " указана более одного
        ↪ раза.\n";
        return 1;
    }

    capitals[state] = cap;
    owner[cap] = state;
    states[state].push_back(cap);
}

int assigned = k;

std::vector<std::priority_queue<Candidate, std::vector<Candidate>,
    ↪ CandidateCmp>> frontiers(k + 1);

auto push_edges = [&](int state, int city) {
    for (const auto& edge : graph[city]) {
        if (owner[edge.to] == 0) {
            frontiers[state].push({edge.len, edge.to, city});
        }
    }
};

for (int state = 1; state <= k; ++state) {
    push_edges(state, capitals[state]);
}

while (assigned < n) {

```

```

bool progress = false;

for (int state = 1; state <= k && assigned < n; ++state) {
    auto& pq = frontiers[state];
    while (!pq.empty() && owner[pq.top().city] != 0) {
        pq.pop();
    }
    if (pq.empty()) {
        continue;
    }

    const Candidate best = pq.top();
    pq.pop();

    owner[best.city] = state;
    states[state].push_back(best.city);
    ++assigned;
    progress = true;

    push_edges(state, best.city);
}

// Для связного графа и корректного входа сюда не попадем,
// но защита помогает избежать заикливания на плохих данных.
if (!progress) {
    std::cerr << "Ошибка: не удалось распределить все города, присвоено
    ↪ только " << assigned
    << " из " << n << ".\n";
    return 1;
}
}

for (int state = 1; state <= k; ++state) {
    std::cout << "Государство " << state << ":";
    for (int city : states[state]) {
        std::cout << ' ' << city;
    }
    std::cout << '\n';
}

return 0;
}

```

7 Демонстрация работы программы

1) Входные данные

Рассматриваемый тест: test1.

Формат входных данных:

1. n, m — число городов и число дорог.
2. Далее m строк вида $i\ j\ len$, где i и j — номера городов, len — длина дороги.
3. k — число государств (число столиц).
4. Далее k чисел — номера столиц.

Ограничения: граф связный, нумерация городов — от 1 до n .

```

6 7
1 2 1
2 3 2
3 4 1
4 5 2
5 6 1
1 6 4
2 5 3
2
1 4

```

2) Вывод

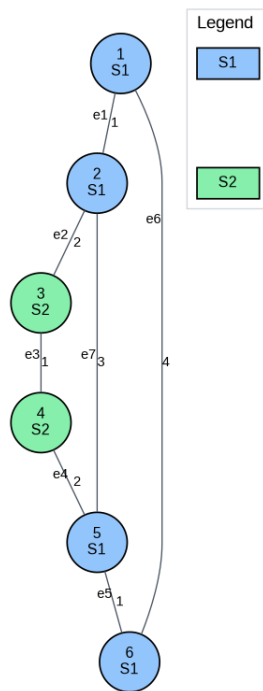
```

artem@artem-BMH-WCX9:~/ТП_6_семестр/ТФЯиТ/Задания/Cities/Solution/Code$ ./cities < tests/test1.in
Государство 1: 1 2 5 6
Государство 2: 4 3

```

3) Визуализация

Cities: test1



4) Описание результата

По результату для данного теста:

- государство 1 получает города {1, 2, 5, 6};
- государство 2 получает города {4, 3}.

Это соответствует пошаговому присоединению ближайших доступных городов при чередовании ходов государств.

5) Дополнительный граничный тест

Для **test3** проверяется ситуация с близкими длинами рёбер и выбором по правилу разрешения равенств:

Входные данные теста **test3**:

```
5 6
1 2 2
1 3 2
2 4 1
3 5 1
4 5 3
2 3 4
2
1 5
```

Государство 1: 1, 2, 4, Государство 2: 5, 3.

Это демонстрирует детерминированность выбора при равных/близких кандидатах.

8 Анализ сложности

Обозначения: n — число городов, m — число дорог, k — число государств.

Построение списков смежности: $O(m)$.

Каждое ребро может быть добавлено в очереди ограниченное число раз (по мере роста состояний), а каждая операция с очередью стоит $O(\log S)$, где S — текущий размер очереди. В сумме время работы:

$$O(m \log m)$$

для типичного и худшего случая данной реализации.

Память:

$$O(n + m)$$

на граф, массивы принадлежности городов и структуры очередей.